

RAPPORT D'EXAMEN

Contrôle – Architecture Distribuée et Middleware

Application de Gestion de Location de Véhicules en Agence

Étudiant	Mustapha ELMIFDALI
Filière	Master SDIA
Email	mustaphamifdali2001@gmail.com
Module	Architecture JEE et Middlewate
Encadrent	Mohamed YOUSSEFI

1. Introduction

Ce rapport présente le développement complet d'une application web de gestion de location de véhicules en agences, réalisé dans le cadre du contrôle du module Architecture Distribuée et Middleware du Master SDIA à l'ENSET Mohammedia.

L'application backend est développée avec Spring Boot 3.3, Spring Security avec JWT, Spring Data JPA et MySQL. Elle expose des APIs REST documentées via Swagger/OpenAPI. Conformément aux instructions de l'examen, seul le backend est couvert dans ce rapport.

2. Analyse des Besoins

2.1 Règles de gestion

- Une agence possède plusieurs véhicules
- Chaque véhicule appartient à une seule agence
- Un véhicule peut être soit une voiture, soit une moto
- Chaque véhicule peut faire l'objet de plusieurs locations
- Chaque location concerne un seul véhicule

2.2 Entités et attributs

Entité	Attributs principaux
Véhicule	id, marque, modele, matricule, prixParJour, dateMiseEnService, statut (Disponible/Loué/EnMaintenance)
Voiture	Hérite de Véhicule + nombrePortes, typeCarburant, boiteVitesse
Moto	Hérite de Véhicule + cylindree, typeMoto, casqueInclus
Agence	id, nom, adresse, ville, telephone
Location	id, dateDebut, dateFin, montantTotal, statut, vehicule, client
Utilisateur	id, nom, prenom, email, motDePasse, role

2.3 Rôles utilisateurs

Rôle	Permissions
ROLE_ADMIN	Accès total : CRUD complet sur toutes les ressources
ROLE_EMPLOYE	Gestion agences, véhicules, validation et suivi des locations
ROLE_CLIENT	Créer une location, consulter ses propres locations

3. Architecture Technique du Projet (Question B.5)

L'application suit une architecture multicouches stricte conforme au pattern JEE :

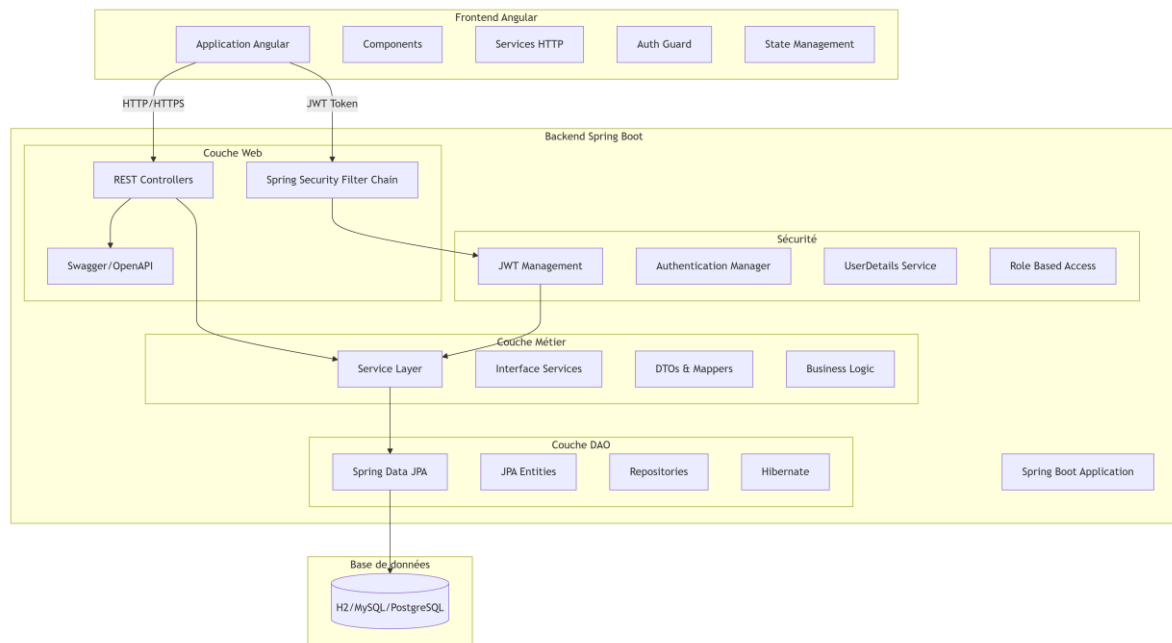


Figure 1 – Architecture technique complète du projet (Frontend Angular + Backend Spring Boot + MySQL)

3.1 Structure du projet

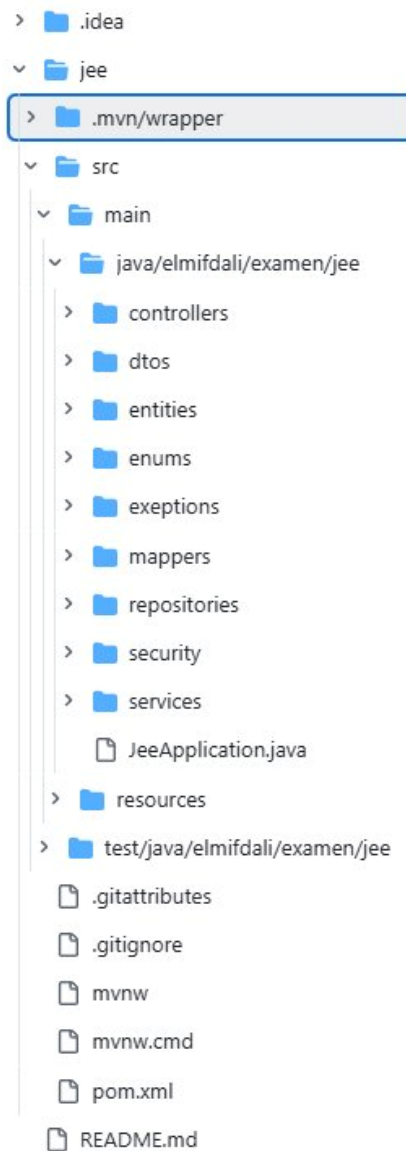


Figure 2 – Structure des packages du projet Spring Boot

3.2 Couches de l'application

Couche	Technologie	Rôle
Présentation (Web)	Spring MVC REST Controllers	Exposition des APIs HTTP/JSON
Sécurité	Spring Security + JWT	Authentification et autorisation par rôle
Métier (Service)	Spring @Service + DTOs	Logique applicative et règles de gestion
DAO	Spring Data JPA + Hibernate	Accès à la base de données
Base de données	MySQL 8.x	Persistance relationnelle
Documentation	SpringDoc OpenAPI 2.5	Swagger UI auto-généré

3.3 Technologies utilisées

Technologie	Version	Rôle
Spring Boot	3.3.0	Framework principal, auto-configuration
Spring Security	6.x	Authentification, autorisation, filtre JWT
JJWT	0.11.5	Génération et validation des tokens JWT
Spring Data JPA + Hibernate	3.3.0 / 6.x	ORM et accès base de données
MySQL	8.x	Base de données relationnelle
SpringDoc OpenAPI	2.5.0	Documentation Swagger UI
Lombok	Latest	Réduction du code boilerplate
Java	21	Langage de développement (LTS)

4. Diagramme de Classes (Question B.6)

Le diagramme ci-dessous présente les entités du domaine avec leurs attributs et leurs relations. L'héritage entre Véhicule, Voiture et Moto est représenté explicitement.

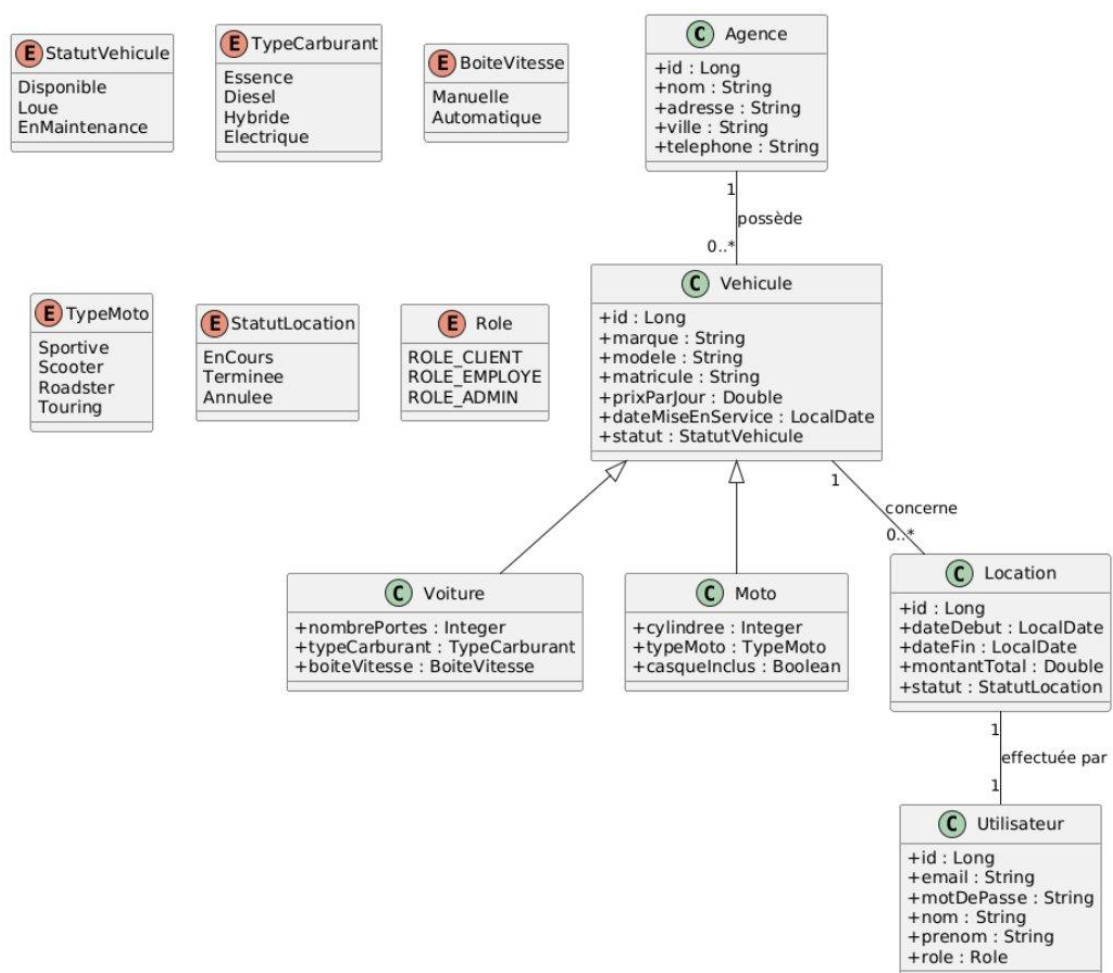


Figure 3 – Diagramme de classes des entités JPA

4.1 Relations entre entités

- Agence → Vehicule : OneToMany (une agence possède plusieurs véhicules)
- Vehicule → Location : OneToMany (un véhicule peut avoir plusieurs locations)
- Vehicule → Voiture / Moto : héritage JOINED (stratégie d'héritage JPA)
- Location → Utilisateur : ManyToOne (une location est effectuée par un client)

5. Couche DAO – Question C.2

5.1 Entités JPA

Entité Vehicule (classe mère)

```
@Entity @Inheritance(strategy = InheritanceType.JOINED)
public class Vehicule {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String marque, modele, matricule;
    private Double prixParJour;
    private LocalDate dateMiseEnService;
    @Enumerated(EnumType.STRING)
    private VehiculeStatus statut = VehiculeStatus.DISPONIBLE;
    @ManyToOne @JoinColumn(name = "agence_id")
    private Agence agence;
    @OneToMany(mappedBy = "vehicule", cascade = CascadeType.ALL)
    private List<Location> locations = new ArrayList<>();
}
```

Entité Voiture

```
@Entity @Table(name = "voiture")
public class Voiture extends Vehicule {
    private int nombrePortes;
    @Enumerated(EnumType.STRING) private TypeCarburant typeCarburant;
    @Enumerated(EnumType.STRING) private BoiteVitesse boiteVitesse;
}
```

Entité Moto

```
@Entity @Table(name = "motorcycle")
public class Motorcycle extends Vehicule {
    private int cylindree;
    @Enumerated(EnumType.STRING) private MotorcycleType typeMoto;
    private boolean casqueInclus;
}
```

Entité Agence

```
@Entity @Table(name = "agence")
public class Agence {
    @Id @Builder.Default
    private String id = UUID.randomUUID().toString();
    private String name, address, city, phone;
    @OneToMany(mappedBy = "agence", cascade = CascadeType.ALL)
    private List<Vehicule> vehicles = new ArrayList<>();
}
```

Entité AppUser

```
@Entity @Table(name = "app_user") @Builder
public class AppUser {
    @Id @Builder.Default
    private String id = UUID.randomUUID().toString();
    private String prenom, nom;
    @Column(unique = true) private String email;
    private String password; // BCrypt
    @Builder.Default private boolean actif = true;
    @ElementCollection(fetch = FetchType.EAGER)
    @Builder.Default private Set<Role> roles = new HashSet<>();
}
```

5.2 Repositories Spring Data JPA

```
public interface VehiculeRepository extends JpaRepository<Vehicule, Long> {
    List<Vehicule> findByStatus(VehiculeStatus status);
    List<Vehicule> findByAgenceId(String agenceId);
    List<Vehicule> findByAgenceIdAndStatus(String agenceId, VehiculeStatus status);
    boolean existsByMatricule(String matricule);
}
```

```
public interface LocationRepository extends JpaRepository<Location, String> {
    List<Location> findByVehiculeId(Long vehiculeId);
    List<Location> findByStatut(RentalStatus statut);
    List<Location> findByEmailClient(String emailClient);
    @Query("SELECT l FROM Location l WHERE l.vehicule.id = :vehiculeId"
        + " AND l.statut IN ('CONFIRMEE', 'EN_COURS') "
        + " AND l.dateDebut <= :dateFin AND l.dateFin >= :dateDebut")
    List<Location> findOverlappingLocations(...);
}
```

5.3 Initialisation des données de test

Un CommandLineRunner dans JeeApplication initialise automatiquement la base de données au démarrage :

- 3 utilisateurs : admin@location.ma / employe@location.ma / client@location.ma
- 2 agences : AutoLoc Casablanca, AutoLoc Rabat
- 3 voitures : Dacia Sandero, Renault Clio, Toyota Corolla
- 1 moto : Yamaha MT-07

6. Couche Service – Question C.3

6.1 DTOs

DTO	Champs principaux
AgenceDTO	id, name, address, city, phone, vehiclesCount
VehiculeDTO (@SuperBuilder)	id, marque, modele, matricule, prixParJour, status, agenceId, type

DTO	Champs principaux
VoitureDTO extends VehiculeDTO	nombrePortes, typeCarburant, boiteVitesse
MotorcycleDTO extends VehiculeDTO	cylindree, typeMoto, casqueInclus
LocationDTO	nomClient, emailClient, dateDebut, dateFin, prixTotal, statut, vehiculeId
AuthResponseDTO	token, type=Bearer, email, nom, prenom, roles
LoginRequestDTO	email, password
RegisterRequestDTO	prenom, nom, email, password

Note : VehiculeDTO, VoitureDTO et MotorcycleDTO utilisent @SuperBuilder (Lombok) pour gérer correctement l'héritage dans le pattern Builder.

6.2 Logique métier – LocationService

- Vérification de l'existence et de la disponibilité du véhicule
- Détection des chevauchements de réservations via requête JPQL
- Calcul automatique du prix total : $\text{prixParJour} \times \text{nombre de jours}$
- Cycle de vie : EN_ATTENTE → CONFIRMEE → EN_COURS → TERMINEE / ANNULEE

6.3 Mappers

Les mappers assurent la conversion entité ↔ DTO sans dépendance externe (pas de MapStruct) :

- AgenceMapper : toDTO(), toEntity(), updateEntity()
- VehiculeMapper : toDTO() avec dispatch automatique Voiture/Motorcycle
- LocationMapper : toDTO() avec infos véhicule incluses

7. REST API et Documentation Swagger – Question C.4

Tous les endpoints REST sont documentés via SpringDoc OpenAPI 2.5 et testables depuis l'interface Swagger UI accessible à : <http://localhost:8088/swagger-ui.html>

7.1 Authentification

Authentification API d'authentification JWT		
POST	/api/auth/register	Inscription utilisateur
POST	/api/auth/login	Connexion utilisateur

Figure 4 – Endpoints d'authentification JWT (register / login)

7.2 Gestion des Agences

Agences Gestion des agences de location			^
DELETE	/api/agences/{id}	Supprimer une agence	🔒 ✓
GET	/api/agences/{id}	Obtenir une agence par ID	🔒 ✓
GET	/api/agences	Lister toutes les agences	🔒 ✓
GET	/api/agences/{id}/vehicules	Lister les véhicules d'une agence	🔒 ✓
POST	/api/agences	Créer une agence	🔒 ✓
PUT	/api/agences/{id}	Modifier une agence	🔒 ✓

Figure 5 – Endpoints CRUD des agences

7.3 Gestion des Véhicules

Véhicules Gestion des véhicules			^
DELETE	/api/vehicules/{id}	Supprimer un véhicule	🔒 ✓
GET	/api/vehicules	Lister tous les véhicules	🔒 ✓
GET	/api/vehicules/{id}	Obtenir un véhicule par ID	🔒 ✓
GET	/api/vehicules/disponibles	Lister les véhicules disponibles	🔒 ✓
PATCH	/api/vehicules/{id}/statut	Changer le statut d'un véhicule	🔒 ✓

Figure 6 – Endpoints de gestion des véhicules

7.4 Gestion des Voitures

Voitures Gestion des voitures			^
GET	/api/voitures/{id}	Obtenir une voiture par ID	🔒 ✓
GET	/api/voitures	Lister toutes les voitures	🔒 ✓
POST	/api/voitures	Créer une voiture	🔒 ✓
PUT	/api/voitures/{id}	Modifier une voiture	🔒 ✓

Figure 7 – Endpoints CRUD des voitures

7.5 Gestion des Motos

Motorcycles Gestion des motos			^
GET	/api/motorcycles/{id}	Obtenir une moto par ID	🔒 ✓
GET	/api/motorcycles	Lister toutes les motos	🔒 ✓
POST	/api/motorcycles	Créer une moto	🔒 ✓
PUT	/api/motorcycles/{id}	Modifier une moto	🔒 ✓

Figure 8 – Endpoints CRUD des motorcycles

7.6 Schémas des DTOs

Schemas		^
VoitureDTO	>	
MotorcycleDTO	>	
AgenceDTO	>	
LocationDTO	>	
RegisterRequestDTO	>	
AuthResponseDTO	>	
LoginRequestDTO	>	
VehiculeDTO	>	

Figure 9 – Schémas Swagger : VoitureDTO, MotorcycleDTO, AgenceDTO, LocationDTO...

7.7 Tableau complet des endpoints

Méthode	URL	Description	Accès
POST	/api/auth/login	Connexion – retourne token JWT	Public
POST	/api/auth/register	Inscription client	Public

Méthode	URL	Description	Accès
GET	/api/agences	Lister / rechercher agences	Public
POST	/api/agences	Créer une agence	ADMIN, EMPLOYE
PUT	/api/agences/{id}	Modifier une agence	ADMIN, EMPLOYE
DELETE	/api/agences/{id}	Supprimer une agence	ADMIN, EMPLOYE
GET	/api/agences/{id}/vehicules	Véhicules d'une agence	Public
GET	/api/vehicules/disponibles	Véhicules disponibles	Public
PATCH	/api/vehicules/{id}/statut	Changer statut véhicule	ADMIN, EMPLOYE
POST	/api/voitures	Créer une voiture	ADMIN, EMPLOYE
PUT	/api/voitures/{id}	Modifier une voiture	ADMIN, EMPLOYE
POST	/api/motorcycles	Créer une moto	ADMIN, EMPLOYE
PUT	/api/motorcycles/{id}	Modifier une moto	ADMIN, EMPLOYE
POST	/api/locations	Créer une location	Authentifié
PATCH	/api/locations/{id}/demarrer	Démarrer la location	ADMIN, EMPLOYE
PATCH	/api/locations/{id}/terminer	Terminer la location	ADMIN, EMPLOYE
PATCH	/api/locations/{id}/annuler	Annuler la location	Authentifié
GET	/api/locations/disponibilite	Vérifier dispo + prix	Public

8. Sécurité Spring Security + JWT – Question C.6

8.1 Flux d'authentification

1. Le client envoie : POST /api/auth/login {email, password}
2. Spring Security valide via AuthenticationManager (BCrypt)
3. JwtService génère un token signé HMAC-SHA256 (expiration 24h)
4. Le token est retourné : {token: "eyJhbGciOiJIUzI1NiJ9..."}
5. Chaque requête suivante : Authorization: Bearer <token>
6. JwtAuthenticationFilter valide le token et charge le UserDetails
7. Spring Security autorise selon les rôles de l'utilisateur

8.2 Configuration Spring Security

```

http.csrf(AbstractHttpConfigurer::disable)
    .authorizeHttpRequests(auth -> auth
        .requestMatchers("/api/auth/**", "/swagger-ui/**").permitAll()
        .requestMatchers(GET, "/api/agences/**", "/api/vehicules/**").permitAll()
        .requestMatchers("/api/agences/**").hasAnyRole("ADMIN", "EMPLOYE")
        .requestMatchers(POST, "/api/locations/**").hasAnyRole("ADMIN", "EMPLOYE", "CLIENT")
        .anyRequest().authenticated()
    )
    .sessionManagement(s -> s.sessionCreationPolicy(STATELESS))
    .addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class);

```

8.3 Génération du token JWT (JJWT 0.11.5)

```
return Jwts.builder()
    .setClaims(extraClaims)
    .setSubject(userDetails.getUsername())
    .setIssuedAt(new Date())
    .setExpiration(new Date(System.currentTimeMillis() + jwtExpiration))
    .signWith(getSigningKey(), SignatureAlgorithm.HS256)
    .compact();
```

8.4 Comptes de test

Email	Mot de passe	Rôle
admin@location.ma	admin123	ROLE_ADMIN
employee@location.ma	employee123	ROLE_EMPLOYEE
client@location.ma	client123	ROLE_CLIENT

9. Base de Données MySQL

9.1 Configuration application.properties

```
spring.datasource.url=jdbc:mysql://localhost:3306/location_agence
    ?createDatabaseIfNotExist=true&useSSL=false
    &serverTimezone=Africa/Casablanca&characterEncoding=UTF-8
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQLDialect
app.jwt.secret=c2VjcmlvLWtleS1lbmlpZmRhbkGktZXhhbWVuLWplZQ==
app.jwt.expiration=86400000
```

9.2 Tables générées

Table	Description
agence	Agences de location (PK : String UUID)
vehicule	Table de base des véhicules (PK : Long AUTO, héritage JOINED)
voiture	Extension voiture (FK → vehicule.id)
motorcycle	Extension moto (FK → vehicule.id)
location	Contrats de location avec calcul de prix
app_user	Utilisateurs avec email unique et password BCrypt
user_roles	Table de jointure utilisateur ↔ rôle

10. Améliorations Additionnelles – Question C.7

- @SuperBuilder Lombok sur la hiérarchie VehiculeDTO/VoitureDTO/MotorcycleDTO pour corriger les conflits builder()
- @Builder.Default sur tous les champs initialisés (id UUID, actif, roles, vehicles)
- GlobalExceptionHandler centralisé : 404, 400, 401, 403, 500 avec messages lisibles
- Validation des entrées : @Valid, @NotBlank, @Email, @Positive sur tous les DTOs

- Calcul automatique du prix total : `prixParJour × ChronoUnit.DAYS.between()`
- Vérification de chevauchement des réservations via requête JPQL dédiée
- Initialisation automatique des données au démarrage (`CommandLineRunner`)
- Transactions `@Transactional(readOnly=true)` sur toutes les opérations de lecture
- Journalisation applicative avec `@Slf4j` sur tous les services
- Encodage UTF-8 forcé + timezone Africa/Casablanca dans `application.properties`

11. Conclusion

Ce projet backend implémente une application complète et professionnelle de gestion de location de véhicules, en répondant à l'ensemble des questions de l'examen :

- Architecture multicouches REST claire et documentée (B.5)
- Diagramme de classes avec héritage Véhicule → Voiture / Moto (B.6)
- Entités JPA avec relations et héritage JOINED (C.2a)
- Repositories Spring Data avec méthodes dérivées et `@Query` (C.2b)
- Initialisation de la base avec données de test (C.2c)
- DTOs, Mappers et Services avec logique métier complète (C.3)
- REST Controllers documentés via Swagger/OpenAPI (C.4)
- Sécurité JWT avec 3 rôles et autorisations par endpoint (C.6)
- Améliorations additionnelles : validation, gestion erreurs, `@SuperBuilder` (C.7)